安装与配置

1、在这个官网链接下载Elasticsearch zip:Deploy an Elasticsearch cluster | Elastic Docs,然后解压到全英文路径

2、以 **管理员身份** 打开 PowerShell 或 CMD：

```
.\bin\elasticsearch.bat
```

控制台里会出现很多配置和安装日志，并且在 **第一次启动时自动完成安全配置**：

- 为内置超级用户 `elastic` 生成一个随机密码；
- 为 HTTP 和节点间通信生成 TLS 证书；
- 自动生成一个 **Kibana enrollment token（注册令牌）**，有效期 30 分钟，用来让 Kibana 自动接入。

非常关键： 控制台里会打印类似（一般在日志的后半部分，通过关键字搜索即可找到）：

```
Password for the elastic user is: XXXXXXXXXXXXX
...
HTTP CA certificate SHA-256 fingerprint is: ...
...
Kibana enrollment token is: YYYYYYYYYYYYY
```

👉 把这几个东西先复制到一个记事本里保存好：

- `elastic` 用户密码（后面 Kibana 登录要用）
- Kibana enrollment token（后面启动 Kibana 要用）

下面是我第一次启动是重要的一些日志：

```
Elasticsearch security features have been automatically
configured!
Authentication is enabled and cluster connections are encrypted.
#用于 elastic 用户的密码（可通过 bin/elasticsearch-reset-password
-u elastic 命令重置）：mEn3ayQc-mn6+6BxkRpS
Password for the elastic user (reset with `bin/elasticsearch-
reset-password -u elastic`):
  mEn3ayQc-mn6+6BxkRpS
#HTTP CA 证书的 SHA-256 指纹：当使用客户端（如 Kibana、浏览器或
curl）连接 Elasticsearch 时，可以用这个指纹来验证您连接的是否是真正
的服务器，而不是恶意的中间人攻击者。这是一种信任凭证
HTTP CA certificate SHA-256 fingerprint:

a9e0b19bffade0bcc5d1aacf32d15078961225ce6c3155615ff1514873910379

Configure Kibana to use this cluster:

Run Kibana and click the configuration link in the terminal when
Kibana starts.
#kibana连接Elasticsearch需要的token(有效时间30分钟)
```

```
Copy the following enrollment token and paste it into Kibana in
your browser (valid for the next 30 minutes):
eyJ2ZXIiOiI4LjE0LjAiLCJhZHIiOlsiMjYuMjYuMjYuMTo5MjAwIl0sImZnciI6ImE5ZTE

Configure other nodes to join this cluster:

On this node:
#token有效期过了之后可以重新使用以下命令再生成一个
token:bin/elasticsearch-create-enrollment-token -s node
Create an enrollment token with `bin/elasticsearch-create-
enrollment-token -s node`.
Uncomment the transport.host setting at the end of
config/elasticsearch.yml.
Restart Elasticsearch.
On other nodes:
#可以使用以下命令在elasticsearch启动时使用自己的token
Start Elasticsearch with `bin/elasticsearch --enrollment-token
<token>`, using the enrollment token that you generated.
```

3、验证Elasticsearch 是否正常

保持 `elasticsearch.bat` 窗口不要关。

在 **主机浏览器** 打开：

```
https://localhost:9200
```

浏览器会弹出安全提示（因为是自签名证书），选“继续访问（不安全）”即可（实验环境）。

然后会让你输入用户名和密码：

- 用户名：`elastic`
- 密码：刚才控制台里生成的那个密码

登录成功后，会看到一个 JSON输出，表示安装成功：

优质打印


```
{
  "name" : "LIZHIXING",
  "cluster_name" : "elasticsearch",
  "cluster_uuid" : "L9Afc2jDTbaLwR7DqpCjhg",
  "version" : {
    "number" : "9.2.1",
    "build_flavor" : "default",
    "build_type" : "zip",
    "build_hash" : "4ad0ef0e98a2e72fafbd79a19fa5cae2f026117d",
    "build_date" : "2025-11-06T22:07:39.673130621Z",
    "build_snapshot" : false,
    "lucene_version" : "10.3.1",
    "minimum_wire_compatibility_version" : "8.19.0",
    "minimum_index_compatibility_version" : "8.0.0"
  },
  "tagline" : "You Know, for Search"
}
```


基础使用

连接（requests + elasticsearch-py，一处配置，全局复用）

In [5]:

```python
import requests
from elasticsearch import Elasticsearch

# ========= 一次性配置（按你的环境改）=========
ES_HTTP = "http://localhost:9200"     # 关闭安全时常见
ES_HTTPS = "https://localhost:9200"   # ES 8.x 默认常见（安全开启）

USE_HTTPS = False        # True: https; False: http
USE_AUTH = True          # 你现在遇到 401，通常应设 True（或去关闭安全）
USERNAME = "elastic"
PASSWORD = "hOpwx=AAnOUKNeu2SBfP"  # 替换为真实密码
VERIFY_CERTS = False     # https + 自签名证书时，学习环境可 False

ES = ES_HTTPS if USE_HTTPS else ES_HTTP


# ========= requests helper: 后面统一用它 =========
def _auth():
    return (USERNAME, PASSWORD) if USE_AUTH else None

def _verify():
    # http 时 verify 没意义；https 时按 VERIFY_CERTS
    return VERIFY_CERTS if USE_HTTPS else True

def es_get(path: str, **kwargs):
    """GET 请求封装：自动拼 URL + 认证 + 证书校验"""
```

```python
    url = ES.rstrip("/") + "/" + path.lstrip("/")
    return requests.get(url, auth=_auth(), verify=_verify(), **kwargs)

def es_put(path: str, **kwargs):
    url = ES.rstrip("/") + "/" + path.lstrip("/")
    return requests.put(url, auth=_auth(), verify=_verify(), **kwargs)

def es_post(path: str, **kwargs):
    url = ES.rstrip("/") + "/" + path.lstrip("/")
    return requests.post(url, auth=_auth(), verify=_verify(), **kwargs)

def es_delete(path: str, **kwargs):
    url = ES.rstrip("/") + "/" + path.lstrip("/")
    return requests.delete(url, auth=_auth(), verify=_verify(), **kwargs)


# ========= 连通性检查 =========
r = es_get("/")
print("status:", r.status_code)
print("keys:", list(r.json().keys()))  # 401 时通常只有 error/status

# ========= elasticsearch-py 客户端（可选）=========
client = Elasticsearch(
    ES,
    basic_auth=(USERNAME, PASSWORD) if USE_AUTH else None,
    verify_certs=_verify()
)

print("client info:", client.info().get("version", {}).get("number"))
```

```
status: 200
keys: ['name', 'cluster_name', 'cluster_uuid', 'version', 'tagline']
client info: 9.2.1
```

索引管理（删/建/查 mapping）

In [6]:

```python
import json

index = "rag_notes"

mapping = {
  "mappings": {
    "properties": {
      "title": {"type": "text"},
      "content": {"type": "text"},
      "tags": {"type": "keyword"},
      "created_at": {"type": "date"}
    }
  }
}

#（学习环境）先删掉旧索引，避免重复执行报错
_ = es_delete(f"/{index}")

# 创建索引
r = es_put(f"/{index}", json=mapping)
print("create:", r.status_code, r.json())

# 查看 mapping
```

```python
r = es_get(f"/{index}/_mapping")
print(json.dumps(r.json(), indent=2, ensure_ascii=False)[:1200])
```

```
create: 200 {'acknowledged': True, 'shards_acknowledged': True, 'index': 'rag_not
es'}
{
  "rag_notes": {
    "mappings": {
      "properties": {
        "content": {
          "type": "text"
        },
        "created_at": {
          "type": "date"
        },
        "tags": {
          "type": "keyword"
        },
        "title": {
          "type": "text"
        }
      }
    }
  }
}
```

写入（单条 + Bulk + refresh）

In [7]:

```python
from datetime import datetime
import json

# 单条写入
doc1 = {
  "title": "RAG 基础：为什么需要检索？",
  "content": "RAG 通过检索外部知识减少幻觉，并提升回答的时效性与可追溯性。",
  "tags": ["rag", "intro"],
  "created_at": datetime.utcnow().isoformat()
}

r = es_post(f"/{index}/_doc", json=doc1)
print("insert one:", r.status_code, r.json().get("result"))

# Bulk 写入：一行 action + 一行 source（NDJSON）
docs = [
  {
    "title": "ElasticSearch 基础：倒排索引",
    "content": "倒排索引把词映射到包含该词的文档列表，从而加速全文检索。",
    "tags": ["elastic", "search"],
    "created_at": datetime.utcnow().isoformat()
  },
  {
    "title": "BM25 简介",
    "content": "BM25 是经典相关性排序算法，综合考虑词频、逆文档频率和文档长度。",
    "tags": ["ranking", "bm25"],
    "created_at": datetime.utcnow().isoformat()
  }
]

bulk_lines = []
```

```
for d in docs:
    bulk_lines.append(json.dumps({"index": {"_index": index}}))
    bulk_lines.append(json.dumps(d, ensure_ascii=False))

payload = "\n".join(bulk_lines) + "\n"

r = es_post(
    "/_bulk",
    data=payload.encode("utf-8"),
    headers={"Content-Type": "application/x-ndjson"}
)
print("bulk:", r.status_code, "errors:", r.json().get("errors"))

# 刷新：让写入尽快可搜索到
_ = es_post(f"/{index}/_refresh")
print("refreshed")
```

```
C:\Users\86177\AppData\Local\Temp\ipykernel_15952\462124077.py:9: DeprecationWarn
ing: datetime.datetime.utcnow() is deprecated and scheduled for removal in a futu
re version. Use timezone-aware objects to represent datetimes in UTC: datetime.da
tetime.now(datetime.UTC).
  "created_at": datetime.utcnow().isoformat()
C:\Users\86177\AppData\Local\Temp\ipykernel_15952\462124077.py:21: DeprecationWar
ning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a fut
ure version. Use timezone-aware objects to represent datetimes in UTC: datetime.d
atetime.now(datetime.UTC).
  "created_at": datetime.utcnow().isoformat()
C:\Users\86177\AppData\Local\Temp\ipykernel_15952\462124077.py:27: DeprecationWar
ning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a fut
ure version. Use timezone-aware objects to represent datetimes in UTC: datetime.d
atetime.now(datetime.UTC).
  "created_at": datetime.utcnow().isoformat()
insert one: 201 created
bulk: 200 errors: False
refreshed
```

代码块 4：查询（match / term / bool + 分页排序 + _source + 高亮）

In [8]:

```
def es_search(body: dict):
    """统一 search：返回完整响应 JSON"""
    r = es_get(f"/{index}/_search", json=body)
    r.raise_for_status()
    return r.json()

# 1) match：全文检索（会分词）
q_match = {"query": {"match": {"content": "倒排 索引"}}}
resp = es_search(q_match)
print("match:", [(h["_score"], h["_source"]["title"]) for h in resp["hits"]["hit

# 2) term：精确匹配（不分词，适合 keyword）
q_term = {"query": {"term": {"tags": "bm25"}}}
resp = es_search(q_term)
print("term:", [h["_source"]["title"] for h in resp["hits"]["hits"]])

# 3) bool：must（参与打分） + filter（不参与打分但更快、可缓存）
q_bool = {
  "query": {
    "bool": {
```

```
      "must": [{"match": {"content": "相关性"}}],
      "filter": [{"term": {"tags": "bm25"}}]
    }
  }
}
resp = es_search(q_bool)
print("bool:", [(h["_score"], h["_source"]["title"]) for h in resp["hits"]["hits

# 4) 分页 + 排序
q_page_sort = {
  "query": {"match_all": {}},
  "from": 0,
  "size": 5,
  "sort": [{"created_at": "desc"}]
}
resp = es_search(q_page_sort)
print("sorted:", [(h["_source"]["created_at"], h["_source"]["title"]) for h in r

# 5) _source：只返回部分字段
q_source = {"query": {"match_all": {}}, "_source": ["title", "tags"], "size": 5}
resp = es_search(q_source)
print("_source:", [h["_source"] for h in resp["hits"]["hits"]])

# 6) 高亮
q_highlight = {
  "query": {"match": {"content": "检索"}},
  "highlight": {"fields": {"content": {}}}
}
resp = es_search(q_highlight)
for h in resp["hits"]["hits"]:
    print("\n-", h["_source"]["title"])
    print("  highlight:", h.get("highlight", {}).get("content", []))
```

```
match: [(3.1076195, 'ElasticSearch 基础：倒排索引'), (0.4676124, 'RAG 基础：为什么需要检索？'), (0.4676124, 'BM25 简介')]
term: ['BM25 简介']
bool: [(2.4192905, 'BM25 简介')]
sorted: [('2026-03-05T15:18:42.360161', 'ElasticSearch 基础：倒排索引'), ('2026-03-05T15:18:42.360161', 'BM25 简介'), ('2026-03-05T15:18:42.313731', 'RAG 基础：为什么需要检索？')]
_source: [{'title': 'RAG 基础：为什么需要检索？', 'tags': ['rag', 'intro']}, {'title': 'ElasticSearch 基础：倒排索引', 'tags': ['elastic', 'search']}, {'title': 'BM25 简介', 'tags': ['ranking', 'bm25']}]

- ElasticSearch 基础：倒排索引
  highlight: ['倒排<em>索</em>引把词映射到包含该词的文档列表，从而加速全文<em>检</em><em>索</em>。']

- RAG 基础：为什么需要检索？
  highlight: ['RAG 通过<em>检</em><em>索</em>外部知识减少幻觉，并提升回答的时效性与可追溯性。']
```

In [9]:

```
### 聚合（按 tags 分组计数）
```

In [10]:

```
q_aggs = {
  "size": 0,
  "aggs": {
    "by_tag": {"terms": {"field": "tags"}}
  }
}
```

```
resp = es_search(q_aggs)
print(resp["aggregations"]["by_tag"]["buckets"])
```

```
[{'key': 'bm25', 'doc_count': 1}, {'key': 'elastic', 'doc_count': 1}, {'key': 'in
tro', 'doc_count': 1}, {'key': 'rag', 'doc_count': 1}, {'key': 'ranking', 'doc_co
unt': 1}, {'key': 'search', 'doc_count': 1}]
```

In [11]:

```
### 常用运维（health / cat / count）
```

In [12]:

```
print("health:", es_get("/_cluster/health").json())

print("\n_cat/indices:")
print(es_get("/_cat/indices?v").text)

print("\n_cat/nodes:")
print(es_get("/_cat/nodes?v").text)

print("\ncount:")
print(es_get(f"/{index}/_count").json())
```

```
health: {'cluster_name': 'elasticsearch', 'status': 'yellow', 'timed_out': False,
'number_of_nodes': 1, 'number_of_data_nodes': 1, 'active_primary_shards': 49, 'ac
tive_shards': 49, 'relocating_shards': 0, 'initializing_shards': 0, 'unassigned_s
hards': 4, 'unassigned_primary_shards': 0, 'delayed_unassigned_shards': 0, 'numbe
r_of_pending_tasks': 0, 'number_of_in_flight_fetch': 0, 'task_max_waiting_in_queu
e_millis': 0, 'active_shards_percent_as_number': 92.45283018867924}

_cat/indices:
health status index
uuid                   pri rep docs.count docs.deleted store.size pri.store.size
dataset.size
green  open   .internal.alerts-transform.health.alerts-default-000001
cyQKIxajSTy7Lfl0SzLfIg   1   0          0            0       249b           249b
249b
green  open   .internal.alerts-observability.logs.alerts-default-000001
Mn_HEd8ZS2KGuIE2oOhJbg   1   0          0            0       249b           249b
249b
green  open   .internal.alerts-observability.uptime.alerts-default-000001
l3pw3sWBR-uVxyV8D4OK_A   1   0          0            0       249b           249b
249b
green  open   .internal.alerts-ml.anomaly-detection.alerts-default-000001
AtpKGgThRoy433HQ9EM0_g   1   0          0            0       249b           249b
249b
green  open   .internal.alerts-observability.slo.alerts-default-000001
us0VKBwESkmu5CXCW-xVAQ   1   0          0            0       249b           249b
249b
green  open   .internal.alerts-default.alerts-default-000001
sh9-o32VTxK_AKDg9pR2LA   1   0          0            0       249b           249b
249b
green  open   .internal.alerts-streams.alerts-default-000001
Uxlrgu-oQoSqsC1a0ZxufA   1   0          0            0       249b           249b
249b
green  open   .internal.alerts-observability.apm.alerts-default-000001
BCbOBwbST32HnOYxpBWBWg   1   0          0            0       249b           249b
249b
green  open   .internal.alerts-security.attack.discovery.alerts-default-000001
1siqIVogTxG1wtN_r_0DRg   1   0          0            0       249b           249b
249b
green  open   .internal.alerts-observability.metrics.alerts-default-000001
6clCEZ2URAiFBhlBJdHu_Q   1   0          0            0       249b           249b
249b
yellow open   rag_notes
SgSF_L2rSAGVxMtFN-ffIg   1   1          3            0      6.8kb          6.8kb
6.8kb
green  open   .internal.alerts-ml.anomaly-detection-health.alerts-default-000001
sGmhyjyuS36qOVs2qGVY6A   1   0          0            0       249b           249b
249b
green  open   .internal.alerts-observability.threshold.alerts-default-000001
tq97FNUrRpuy0CS-NTz4Mw   1   0          0            0       249b           249b
249b
yellow open   openedr-2026.01.07
EtyYmbfASoS926NmUk2Bcw   1   1       1507            0      1.9mb          1.9mb
1.9mb
green  open   .internal.alerts-security.alerts-default-000001
Xv0VziciRHeRa-s1Hy23kA   1   0          0            0       249b           249b
249b
green  open   .internal.alerts-dataset.quality.alerts-default-000001
1Yk7_V21Sl6sIl3gsdXqIQ   1   0          0            0       249b           249b
249b
yellow open   openedr-2025.11.22
```

```
65TI0Rb1QxCUf--EI4n54g   1   1        549            0    474.7kb        474.7kb
474.7kb
yellow open   openedr-2026.01.06
mtuM0CDATkG6FaHxCyZ6EA   1   1       4136            0      1.5mb          1.5mb
1.5mb
green  open   .internal.alerts-stack.alerts-default-000001
56b4Z9gZSuemiNptt4H60w   1   0          0            0       249b           249b
249b


_cat/nodes:
ip        heap.percent ram.percent cpu load_1m load_5m load_15m node.role   maste
r name
127.0.0.1            7          83   2                          cdfhilmrstw *
LIZHIXING


count:
{'count': 3, '_shards': {'total': 1, 'successful': 1, 'skipped': 0, 'failed': 0}}
```

In [ ]: